

Mobility-Induced Graph Learning for WiFi Positioning

Kyuwon Han^{ID}, Seung Min Yu^{ID}, Seong-Lyun Kim^{ID}, and Seung-Woo Ko^{ID}, *Senior Member, IEEE*

Abstract—A smartphone-based user mobility tracking could be effective in finding his/her location, while the unpredictable error therein due to low specification of built-in *inertial measurement units* (IMUs) rejects its standalone usage but demands the integration to another positioning technique like WiFi positioning. This paper aims to propose a novel integration technique using a graph neural network called *Mobility-Induced Graph Learning* (MINGLE), which is designed based on two types of graphs made by capturing different user mobility features. Specifically, considering sequential *measurement points* (MPs) as nodes, a user's regular mobility pattern allows us to connect neighbor MPs as edges, called *time-driven mobility graph* (TMG). Second, a user's relatively straight transition at a constant pace when moving from one position to another can be captured by connecting the nodes on each path, called a *direction-driven mobility graph* (DMG). Then, we can design *graph convolution network* (GCN)-based cross-graph learning, where two different GCN models for TMG and DMG are jointly trained by feeding different input features created by WiFi RTTs yet sharing their weights. Besides, the loss function includes a mobility regularization term such that the differences between adjacent location estimates should be less variant due to the user's stable moving pace. Noting that the regularization term does not require ground-truth location, MINGLE can be designed under semi- and self-supervised learning frameworks. The proposed MINGLE's effectiveness is extensively verified through field experiments, showing a better positioning accuracy than benchmarks, say *mean absolute errors* (MAEs) being 1.510 (m) and 1.077 (m) for self- and semi-supervised learning cases, respectively.

Index Terms—WiFi positioning, graph neural network, mobility-induced graph, graph convolution network, cross-graph learning, mobility-regularization term.

I. INTRODUCTION

With the widespread use of smartphones and ubiquitous WiFi *access points* (APs), WiFi positioning has been considered a popular solution for providing user-specific services by identifying the user's location [1]. The distances between a target user and each WiFi AP can be estimated based on *signal strength* (SS) or *round trip time* (RTT), allowing us to find the location estimate through multilateration. However, its accuracy and reliability are questionable since these

measurements can be severely corrupted unless the *line-of-sight* (LoS) condition is made between the user and each AP. One viable approach to overcome the limitation is that the user's mobility information measured by a built-in *inertial measurement unit* (IMU) is involved in WiFi positioning since it is independent of the propagation conditions affecting SS or RTT. Although various techniques to integrate user mobility into WiFi positioning have been proposed in the literature introduced in the sequel, this paper aims to propose a novel technique using a *graph neural network* (GNN), called *Mobility-Induced Graph Learning* (MINGLE), which is based on graphs created by capturing user-mobility inherent features. Specifically, a user's regular mobility patterns, such as constant moving speed and directions that are detectable by the user's smartphone's IMUs, enable us to generate graphs in real time. Nodes represent the user's locations, and the edges between the nodes are created depending on their temporal adjacency or direction consistency. The generated graphs are cores of the proposed MINGLE. The input features (raw or pre-processing data) are propagated to the following layers' nodes only when the corresponding edges exist. Besides, a regular inter-distance between adjacent nodes can be used as a regularization term in designing a loss function, making it possible to develop MINGLE under semi-supervised and self-supervised learning frameworks.

A. Prior Work

We summarize relevant works in the literature, which can be divided into two parts: WiFi positioning with user mobility and GNN-based positioning.

1) *WiFi Positioning with User Mobility*: Recent smartphones equip various IMU sensors, e.g., an accelerometer, gyroscope, and magnetometer, to help update the user's location in GPS-denied environments, called *pedestrian dead reckoning* (PDR) [8]. On the other hand, a standalone PDR approach is hardly considered due to several practical limitations, such as the low resolution of the built-in IMUs, accumulated error over time [9], and time-varying holding patterns [10]. It calls for integrating another positioning approach as a remedy to overcome the above limitations, e.g., WiFi positioning as a main target of this work.

The primary focus of research in this field is to create a method that combines WiFi positioning and PDR. One classic way is to use *Kalman filter* (KF) or *extended KF* (EKF) where two different location estimates of WiFi positioning and PDR, termed *prediction location estimate* (PLE) and *measurement location estimate* (MLE) respectively, are combined by weight-

This work was supported in part by the National Research Foundation of Korea (NRF) funded by the Ministry of Science and ICT (MSIT) (No. 2022R1F1A1072911 and No. 2022R1A5A1027646), the Institute of Information & Communications Technology Planning & Evaluation (IITP) funded by MSIT (No. 2021-0-00347), and a grant from R&D Program (PK2401B2) of the Korea Railroad Research Institute.

K. Han is with Advanced R&D Team, Samsung Electronics, Suwon 16677, Korea (email: kyuwon.han@samsung.com). S. M. Yu is with Innovative Transportation and Logistics Research Center, Korea Railroad Research Institute, Uiwang 16105, Korea (email: smyu@krrri.re.kr). S.-L. Kim is with Dept. EEE, Yonsei University, Seoul 03722, Korea (email: slkim@ramo.yonsei.ac.kr). S.-W. Ko is with Dept. Smart Mobility Eng., Inha University, Incheon 21999, Korea (email: swko@inha.ac.kr). (Corresponding author: Seung-Woo Ko.)

TABLE I
SURVEY OF GNN-BASED POSITIONING METHODS.

Reference	Node	Edge	Feature	GNN	Loss function	Output
[2]	APs	Fully-connected (1/distance)	SS	GCN	Cross-entropy	Fingerprint label
[3]	APs	Fully-connected (1/distance)	SS	GCN	MSE	2D location
[4]	APs, target	Fully-connected (1/distance)	SS	GraphSAGE	Huber	2D location
[5]	APs	Fully-connected (1/distance)	SS	GIN	MAE	2D location
	Clients	Fully-connected (Distribution similarity)	Embedding vector	GCN		Set of local weights
[6]	APs	Inter-distance (Binary)	Estimated distance	GCN	RMSE	2D location
[7]	Pixels	Neighbor pixels (Binary)	Amplitude and phase of CSI	GraphSAGE	Cross-entropy	Fingerprint label
MINGLE	Sequence of MPs	Temporal adjacency, direction consistency (Binary)	RTTs	GCN	MSE	2D location

ing a more reliable one [11]. A prerequisite for KF- or EKF-based integration is that each location estimate's covariance matrix should be given in prior, yet challenging in practice. Several studies have been made to cope with this issue. In [12] and [13], these covariance matrices are computed by leveraging the power of *deep neural network* (DNN), assuming that the ground-truth covariance matrix is given for DNN training. In [14], a novel technique called *combinatorial data augmentation* (CDA) is proposed for updating the MLE's covariance matrix without a ground-truth one such that spatial variance of multiple location estimates obtained from different subsets of WiFi RTT measurements is considered a real-time covariance matrix of MLE. Note that the above techniques are designed based on the belief that KF provides an optimal fusion between PLE and MLE, which only makes sense when the errors on PLE and MLE follow zero-mean Gaussian distributions. Besides, KF targets to fuse a pair of single PLE and MLE, making it difficult to distill useful information embedded in the previous results.

Unlike the above KF-based integration between a single PLE and MLE, a few recent studies aim to exploit the entire sequences of MLEs and PLEs. In [15], the above two sequences are aligned with the minimum error, called *trajectory alignment* (TA), by jointly optimizing WiFi NLoS bias, step length, and initial heading direction. It is extended into the DNN-based algorithm in [16] that the loss function is defined as the difference between the two sequences. On the other hand, these two algorithms work well under the assumption of a relatively precise IMU measurement, necessitating precise calibration and pre-processing to this end [17].

2) *GNN-Based Positioning*: GNN is a class of DNN that uses graph structures to extract domain knowledge therein [18]. While conventional DNNs open all nodes to connect in the following layers, GNN compels each node to update its latent states based on the aggregated information from its adjacent nodes, effectively learning local features in large-scale networks. Traditionally, GNN has been applied to cases when a graph structure is innately given (e.g., social networking [19]). Several recent studies attempt to apply GNN to more general cases by creating a graph from the problem formulation, such

as radio resource optimization in wireless networks [20].

Radio-based positioning is another emerging application that applies GNN, the central theme of this work. A straightforward way to create a graph is by designating positioning APs as nodes. Assuming that the APs' locations are given, the edges between them can be constructed based on their geometric relation. For example, a fully connected graph is created in [2] and [3] whose edge weights are assigned inversely proportional to the corresponding inter-distances. The graph is used to construct a positioning algorithm comprising several *graph convolution network* (GCN) layers, GNN version of convolution networks [21]. In [4], three types of APs (WiFi, ZigBee, and Bluetooth) are considered, each modeled as a fully connected bidirectional graph. The directional edge between a user and an AP is then added, with a weight inversely proportional to the estimated distance. With the aid of GraphSAGE layer [22], it is possible to localize the user by unifying all generated graphs. The scenarios with many users are considered in [6], where both APs and users are designated nodes, assuming every node can communicate.

Several recent studies have suggested different ways to configure a graph other than the above approach. In [7], *channel state information* (CSI) is converted into a 2D image made up of complex numbers. Pixels in the image are treated as nodes, and their adjacency determines connectivity. The complex number's magnitude and phase become an input feature of the constructed graph. In [5], GNN-based localization is extended into federated learning, where the global server regards clients as nodes and determines an adjacency matrix between them by utilizing the similarity between different clients' embedding vectors. Each client updates its model based on *graph isomorphism network* (GIN) [18], and the server aggregates them via GCN with the self-attention mechanism. We have summarized the GNN-based positioning techniques discussed above in Table I.

Despite their effectiveness, most GNN-based positioning techniques share one practical limitation: they are designed based on supervised learning. They require labeling many data samples, usually ground-truth locations at which the corresponding measurements are collected, which is a time-

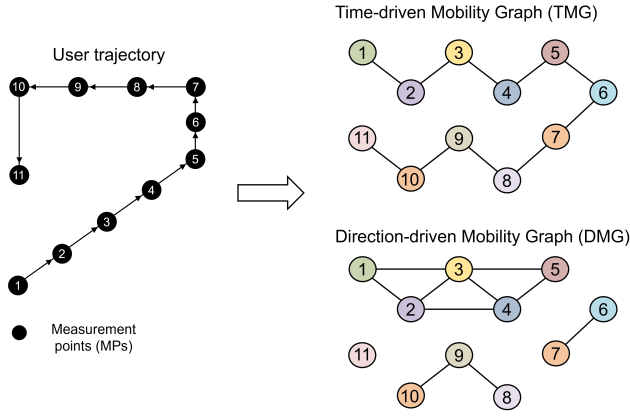


Fig. 1. Graphical example representing the relation between a user's real trajectory and two types of mobility-induced graphs, namely, TMG and DMG defined in Definition 2.

consuming and labor-intensive campaign. Besides, even if data labeling is given, the trained GNN's validity duration is limited because radio signals' features change over time due to various factors, e.g., temperature, pedestrian density, and so on, leading to a mismatch between training data and actual measurements.

B. Contributions

Contrary to the works mentioned above, we exploit user mobility as a critical ingredient to design MINGLE. Specifically, we focus on user mobility's inherent features: people tend to move at a relatively constant speed determined by their mobility types (e.g., walking, running, and so on) [23] and make a straight transition while moving from one to another place [24]–[26]. Due to a smartphone's constant sampling interval, the displacement between adjacent *measurement points* (MPs) usually remains invariant, offering the following three-fold opportunities. First, considering the spatial correlation of WiFi measurements, we can create a mobility-induced graph by setting MPs as nodes and connecting them when they are temporally adjacent. We call it a *time-driven mobility graph* (TMG) throughout the work. Next, every MP on the user's straight transition, which is enough to be detectable using built-in IMUs, can be connected to create another mobility-induced graph, called a *direction-driven mobility graph* (DMG). Figure 1 shows a graphical illustration of TMG and DMG. As the user's trajectory becomes longer, the degrees of TMG and DMG increase, helping enhance the positioning performance. Last, such a regular displacement between neighbor MPs can be reflected in the loss-function design to train a GNN without the ground-truth location. In other words, exploiting user mobility makes it possible to develop MINGLE under the frameworks of semi-supervised learning and self-supervised learning, overcoming the main drawback of the state-of-the-art GNN positioning works.

The main contributions of this work are given below.

- **Real-Time Generation of Mobility-Induced Graphs:** As mentioned before, two types of graphs, say TMG and DMG, are generated based on the real-time movement

of the user, each effectively extracting distinct features that are useful when analyzing the user's mobility pattern. First, TMG is a connected graph that represents the inter-distance between MPs depending on the number of hops between the corresponding nodes. Second, DMG is a disconnected graph comprising multiple sub-graphs, each representing the user's straight-moving pattern. It helps express the directional consistency of MPs on each sub-graph.

- **Cross-Graph Learning and Mobility Regularization of MINGLE Architecture:** MINGLE has two signature design components: Cross-graph learning and mobility regularization. First, it is essential to extract different potentials of TMG and DMG and make a unified framework for estimating the user's location precisely. To this end, we design MINGLE based on cross-graph learning such that two different GNNs are individually trained with different types of input features while their weights are shared. Second, the user's stationary pace can be reflected on the loss function's regularization term, which is expressed as a small variance of inter-distances between adjacent MPs. As aforementioned, the mobility regularization does not require ground-truth locations and helps find precise location estimates of the user when a few or no ground-truths are given.
- **Extensive Field Experiment:** We conducted an extensive field experiment covering different mobility patterns and temporal environment changes. The proposed MINGLE's effectiveness is well verified by showing a much more accurate positioning result than several benchmarks, say the *mean absolute errors* (MAEs) being 1.510 and 1.077 (m) without and with 5-10% ground-truth locations, respectively.

The rest of the paper is organized as follows. The system model and problem formulation are presented in Section II. The proposed mobility-induced graph learning framework is presented in Section III, including the definition of mobility-induced graphs, input feature generation, network architecture, and loss function design. Experimental results are presented in Section IV, followed by concluding remarks in Section V.

II. SYSTEM MODEL AND PROBLEM FORMULATION

This section explains our system model, including a network model and types of measurements. Then, we describe the problem we tackle throughout the work.

A. Network Model

Consider a WiFi positioning network in which M WiFi APs are deployed. The set of APs' indices is denoted by $\mathcal{M} = \{1, 2, \dots, M\}$. Each AP's *two-dimensional* (2D) coordinates, denoted by $\mathbf{z}^{(m)} \in \mathbb{R}^2$ for AP $m \in \mathcal{M}$, are assumed to be stationary and given in advance. Next, a smartphone user moves around the network, and several measurements explained in the sequel are sampled with the constant interval of Δ (sec). The locations where the measurements are sampled are referred to as MPs. The sequence of the MPs' coordinates is given as $\{\mathbf{x}_n\}$, where $\mathbf{x}_n \in \mathbb{R}^2$ is the n -th

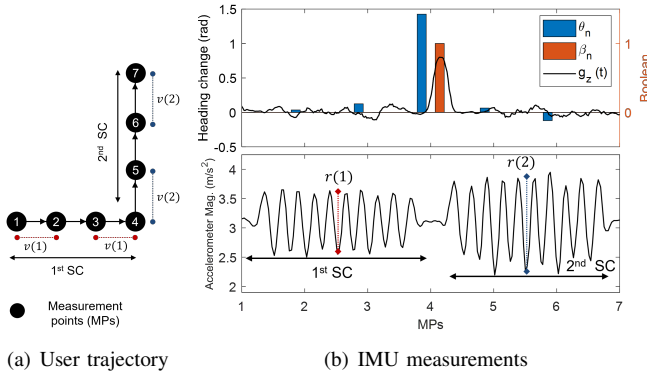


Fig. 2. The relation between user mobility trajectory and the resultant IMU measurements. (a) An example of user trajectory with 7 MPs and 2 SCs. (b) The accelerometer and gyroscope's measurements and their relevant metrics, including raw heading change θ_n (2), binary heading change β_n (3), and the average gap between accelerometer's peaks and valleys $r(\ell)$ specified in (5).

MP's 2D coordinates. The set of MPs' indices is denoted by $\mathcal{N} = \{1, \dots, N\}$, where N is the number of MPs within the duration of T (sec), given as $N = \lfloor \frac{T}{\Delta} \rfloor$ with $\lfloor \cdot \rfloor$ being a floor function. Except for a few MPs that coincide with landmarks in the concerned area (e.g., pillars and gates), it is reasonable to consider the MP's coordinates to be generally unknown. We introduce a binary sequence $\{\alpha_n\}_{n=1}^N$ where α_n is one if the corresponding ground truth $\mathbf{x}_n$ is known while zero otherwise, say $\alpha_n \in \{0, 1\}$. The index set of MPs without ground truths is defined as $\mathcal{X} = \{n | \alpha_n = 0, n \in \mathcal{N}\}$. The objective of this work is to precisely estimate the coordinates of the MPs in $\mathcal{X}$.

B. Measurements and Assumption

The user's smartphone can collect various measurements to help localize the user. Among them, this work considers two kinds of measurements, namely RTT and heading direction change, each of which will be elaborated as follows.

1) *Round Trip Time*: An RTT between each WiFi AP and the smartphone, defined as double propagation time between the two, is measurable with the *fine-timing measurement* (FTM) protocol initiated from IEEE 802.11mc [27]. We assume that both WiFi APs and the user's smartphone support the FTM protocol, and all APs' RTTs can be estimated at every sampled MP, denoted by $\tau_n \in \mathbb{R}^M$ given as

$$\tau_n = [\tau_n^{(1)}, \tau_n^{(2)}, \dots, \tau_n^{(M)}], \quad n = 1, \dots, N, \quad (1)$$

where $\tau_n^{(m)}$ is the RTT measurement from WiFi AP m at the n -th MP.

With a LoS condition, the inter-distance between WiFi AP and the user is precisely detectable by multiplying the half of the light speed $c \approx 3 \times 10^8$ (m/s) to the corresponding RTT, as experimentally verified in [28]. On the other hand, in the case of an NLoS condition, a WiFi signal's propagation path is severely deviated from the direct path due to frequent blockages, resulting in overestimating the inter-distance [29]. Unlike several prior works addressing the NLoS issue by stochastic approaches (e.g., [30] and [31]), there are no statistics on LoS and NLoS conditions.

2) *Heading Direction Change*: A smartphone's built-in gyroscope sensor can track its heading direction change. Consider an instant when the user is located at the n -th MP. During the movement to the $(n+1)$ -th MP, the smartphone has recorded the gyroscope's z-axis measurement, denoted by $g_z(t)$ with $n\Delta < t \leq (n+1)\Delta$. Then, the corresponding heading change, denoted by θ_n , can be computed by integrating $g_z(t)$ as

$$\theta_n = \int_{n\Delta}^{(n+1)\Delta} g_z(s) ds, \quad n = 1, \dots, N-1. \quad (2)$$

It is well known that the gyroscope's measurement is sensitive to the user's unpredictable activity, bringing about considerable bias and noise. The resultant heading change could be significantly different from the ground truth. Instead of the noisy raw heading direction θ_n , we quantize it into two states representing whether the direction is significantly changed. Specifically, we introduce a sequence of binary variable $\{\beta_n\}_{n=1}^N$, where β_n is the quantization value of θ_n , given as

$$\beta_n = \begin{cases} 1, & \text{if } \theta_n \geq \delta, \\ 0, & \text{otherwise,} \end{cases} \quad (3)$$

where δ is a predefined threshold. Without loss of generality, we set $\beta_N = 1$ to represent the end of the trajectory. We set δ as 0.5 (rad) in the field experiment in Sec. IV. Fig. 2 graphically illustrates the relation between raw heading change θ_n and its quantization value β_n . With $\{\beta_n\}$, we can make the following definition:

Definition 1 (Steady Course). A user is said to be on a *steady course* (SC), which is defined as the sub-sequence of consecutive MPs whose β_n therein are all zero except the last one. Specifically, the ℓ -th SC can be expressed in terms of the sub-sequence $\{s(\ell), s(\ell)+1, \dots, e(\ell)-1, e(\ell)\}$, satisfying the following condition:

$$\{\beta_{s(\ell)}, \beta_{s(\ell)+1}, \dots, \beta_{e(\ell)-1}, \beta_{e(\ell)}\} = \{0, 0, \dots, 0, 1\}, \quad (4)$$

where $s(\ell)$ and $e(\ell)$ are the first and the last MPs' indices on the ℓ -th SC, respectively.

Assumption 1 (Consistent Speed). The user's moving speed is assumed to be constant on the same SC, while it may be changed over different SCs.¹

Assumption 1 allows us to consider the MPs on the same SC equally separated. The relation of MPs' inter-distance on different SCs will be explained in the following part.

3) *Speed Variation*: A smartphone's accelerometer helps estimate how much its user accelerates or decelerates his/her pace when changing SCs. Denote the accelerometer's three-dimensional measurement $\mathbf{u}(t) \in \mathbb{R}^3$, whose L2-norm $\|\mathbf{u}(t)\|$ repeatedly oscillates between peaks and valleys when moving on the ℓ -th SC, as shown in Fig. 2. By following several prior

¹As mentioned in Sec. I, a user tends to move from one to another place in a consistent moving pattern (walking, running, and so on) with a stationary pace, allowing many works in the literature to use Assumption 1 (see, e.g., [24], [25], and [26]).

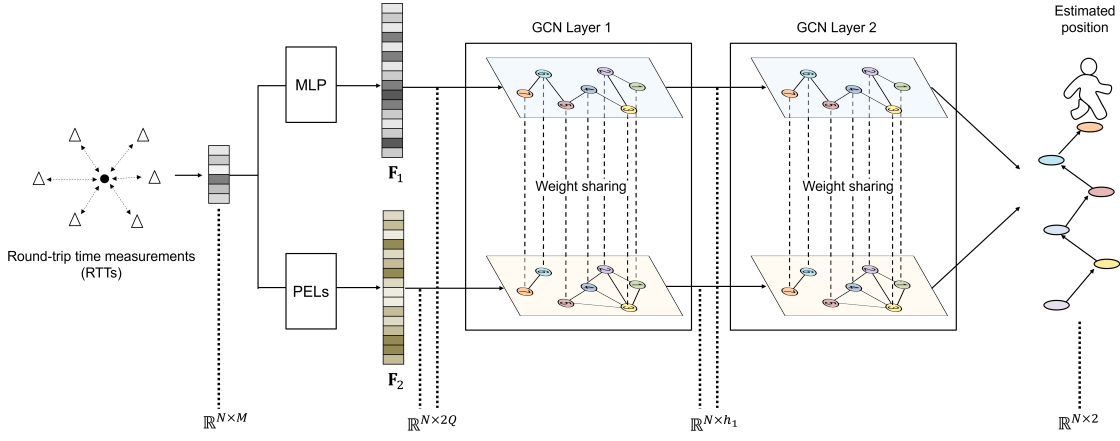


Fig. 3. Graphical illustration of MINGLE architecture. First, the sequence of RTT vectors, say $\{\tau_n\}$ of (1), is used to generate input features through post-processing techniques specified in the sequel. Next, the input features are embedded on two types of mobility-induced graphs defined in Definition 2 through a two-layer GCN with weight sharing. Last, the output of the GCN is considered the 2D location estimates of all MPs.

studies, e.g., [8] and [32], the average gap between peaks and valleys within the same SC, denoted by $r(\ell)$, is considered inversely proportional to the fourth power of the step length. Noting that the step length is proportional to the corresponding moving speed, the moving speed on the ℓ -th SC can be given as $\eta \cdot r(\ell)^{\frac{1}{4}}$ where η depends on the user's personal parameter unknown in practice. Then, we can obtain the speed variation between different SCs by normalizing it with their minimum value, given as

$$v(\ell) = \frac{r(\ell)^{\frac{1}{4}}}{\min_s [r(s)^{\frac{1}{4}}]}, \quad (5)$$

which is independent of η .

C. Problem Description

Given $\{\tau_n\}$ of (1), $\{(s(\ell), e(\ell))\}$ of (4), and $\{v(\ell)\}$ of (5), we attempt to estimate the user's location in semi-supervised or self-supervised learning manner. Specifically, the sequence of the location estimates is denoted by $\{\hat{\mathbf{x}}_n\}$, where $\hat{\mathbf{x}}_n$ represents the location estimate at the n -th MP. As recalled in Sec. II-A, the coordinates of all APs, say $\mathbf{z}^{(m)}$ for all $m \in \mathcal{M}$, are known in advance. Conditioned on the above information, we target to find a mapping function f defined as

$$\{\hat{\mathbf{x}}_n\} = f\left(\{\tau_n\}, \{(s(\ell), e(\ell))\}, \{v(\ell)\} \mid \{\mathbf{z}^{(m)}\}\right), \quad (6)$$

which is constrained by the following three requirements:

1) *Ground-Truth Location*: The location estimates at the MPs not in $\mathcal{X}$ should be equivalent to the ground truth, namely,

$$\hat{\mathbf{x}}_n = \mathbf{x}_n, \quad \forall n \in \mathcal{X}^c. \quad (7)$$

Unless the complementary set $\mathcal{X}^c$ is empty, the concerned positioning problem becomes semi-supervised learning. Otherwise, it is reduced to self-supervised learning.

2) *Equivalent Inter-Distance within one SC*: By Assumption 1, the location estimates of adjacent MPs on the same SC should be equally separated, namely,

$$\|\hat{\mathbf{x}}_{n-1} - \hat{\mathbf{x}}_n\| = \|\hat{\mathbf{x}}_n - \hat{\mathbf{x}}_{n+1}\|, \quad s(\ell) \leq n < e(\ell), \quad n \geq 2, \quad (8)$$

where $s(\ell)$ and $e(\ell)$ are the first and last MPs' indices on the ℓ -th SC specified in (4).

3) *Inter-Distance Change over SCs*: As recalled in Sec. II-B3, the inter-distance between adjacent MPs on the ℓ -th SC is proportional to $v(\ell)$ of (5), leading to the following condition:

$$\frac{\|\hat{\mathbf{x}}_{n_1-1} - \hat{\mathbf{x}}_{n_1}\|}{v(\ell_1)} = \frac{\|\hat{\mathbf{x}}_{n_2-1} - \hat{\mathbf{x}}_{n_2}\|}{v(\ell_2)}, \quad (9)$$

where $s(\ell_1) \leq n_1 \leq e(\ell_1)$ and $s(\ell_2) \leq n_2 \leq e(\ell_2)$ for $n_1 \geq 2$ and $n_2 \geq 2$.

III. MOBILITY-INDUCED GRAPH LEARNING

This section proposes a novel GNN-based positioning algorithm, called MINGLE, to address the problem mentioned in Sec. II-C. We first introduce two types of graphs induced by user mobility and overview the entire architecture of MINGLE based on the generated graphs. Next, we will explain each part of MINGLE in detail, including input feature generation, network architecture, and loss function.

A. Mobility-Induced Graphs and Architecture Overview

A graph can be defined when nodes and edges are specified. In the following definition, two generated graphs, say TMG and DMG, will be defined from the perspective of their nodes and edges.

Definition 2 (Mobility-Induced Graphs). TMG and DMG share the same nodes as all N MPs' indices $\mathcal{N}$. On the other hand, the edges of TMG and DMG depend on adjacent matrices $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{N \times N}$, respectively, as specified below:

- **TMG**: Two nodes in TMG are linked if the difference between the two indices is smaller than the predefined threshold ϵ , given as

$$\mathbf{A}_{i,j} = \begin{cases} 1, & \text{if } |i - j| \leq \epsilon, \\ 0, & \text{otherwise.} \end{cases} \quad (10)$$

- **DMG**: The edges of DMG depend on the SC defined in Definition 1. Assume there are L SCs, each of which

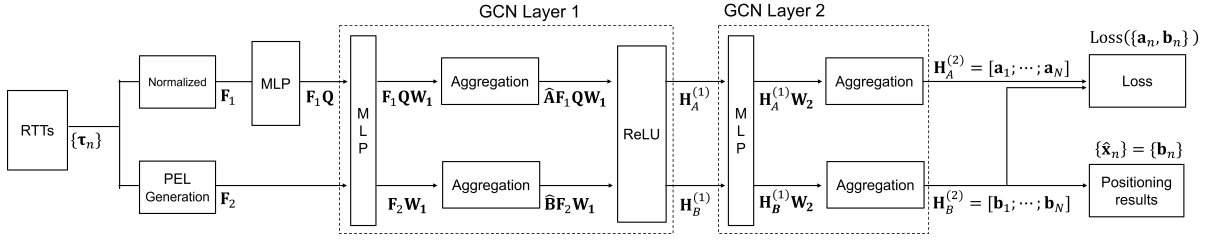


Fig. 4. Two-layer GNN architecture of MINGLE, each of which the layer comprises two GCNs for TMG and DMG with sharing their weights. The detail form of the loss function $\text{Loss}(\{a_n, b_n\})$ is specified in (27).

the length is given as $d(\ell) = e(\ell) - s(\ell) + 1$. Then, two nodes of DMG are linked if their difference is less than the half-length², given as $B = \sum_{\ell=1}^L B(\ell)$, where

$$(B(\ell))_{i,j} = \begin{cases} 1, & \text{if } i, j \in \{s(\ell), \dots, e(\ell)\} \\ & \text{and } |i - j| \leq \lfloor \frac{d(\ell)}{2} \rfloor, \\ 0, & \text{otherwise.} \end{cases} \quad (11)$$

As a result, TMG and DMG can be represented as $G_A(\mathcal{N}, \mathcal{E}_A)$ and $G_B(\mathcal{N}, \mathcal{E}_B)$ respectively, where

$$\mathcal{E}_A = \{(i, j) | A_{i,j} = 1\}, \quad \mathcal{E}_B = \{(i, j) | B_{i,j} = 1\}. \quad (12)$$

Remark 1 (TMG and DMG). The two generated graphs can contribute differently to estimating the user's unknown locations. TMG effectively captures the spatial correlation between neighbor MPs sampled at similar times, helping every location estimate connected to its neighbor ones. On the other hand, DMG focuses on the user's linear movements, relieving the burden of localizing the MPs thereon by reducing the concerned searching dimensions.

To fully unleash the distinct potentials of TMG and DMG, we develop the architecture of MINGLE based on them, as illustrated in Fig. 3. Each part of the architecture will be elaborated in the following subsections.

B. Input Feature Generation

We use two types of input features. The first is the matrix stacking normalized RTT vectors, denoted by $F_1 \in \mathbb{R}^{N \times M}$, given as

$$F_1 = \begin{bmatrix} \frac{\tau_1}{\tau_1 \mathbf{1}}; \frac{\tau_2}{\tau_2 \mathbf{1}}; \dots; \frac{\tau_N}{\tau_N \mathbf{1}} \end{bmatrix}, \quad (13)$$

where $\mathbf{1}$ is a column vector whose elements are all 1.

Second, motivated by the CDA approach in [14], we augment the above input feature through different RTT combinations. For example, consider M RTTs at the n -th MP, say τ_n of (1). We can choose K among M ones, enabling us to make $Q = \binom{M}{K}$ different RTT sets. When $K \geq 3$, each set's RTTs can be transformed into a 2D position estimate using a conventional multilateration algorithm (e.g., [33]), denoted by $y^{(q)} \in \mathbb{R}^{1 \times 2}$. We call it a *preliminary estimated*

location (PEL). We make the augmented input feature vector $f_n \in \mathbb{R}^{1 \times 2Q}$ by concatenating all PELs, given as

$$f_n = [y^{(1)}, y^{(2)}, \dots, y^{(Q)}]. \quad (14)$$

Then, the second input feature matrix, denoted by $F_2 \in \mathbb{R}^{N \times 2Q}$ can be derived by stacking the normalized version of f_n , given as

$$F_2 = \begin{bmatrix} \frac{f_1}{f_1 \mathbf{1}}; \frac{f_2}{f_2 \mathbf{1}}; \dots; \frac{f_N}{f_N \mathbf{1}} \end{bmatrix}. \quad (15)$$

The effectiveness of PEL is well verified in [14] such that when PELs are used as input features for finger-print positioning, positioning accuracy is significantly improved compared to the case with input features being RTTs.

Remark 2 (Low-Dimensional vs. High-Dimensional Features). Noting that the number of PELs Q is much larger than the number of APs M , two matrices F_1 and F_2 can be considered the input features embedded in low-dimensional and high-dimensional spaces, respectively, which play complementary roles in the proposed GNN-based positioning algorithm. GNN is known to be effective when large-scale input features are given, like F_2 . Nevertheless, the augmentation to a high dimensional space may diminish the spatial correlation between neighbor features. We use F_1 as a supplementary to address the limitation, which is low-dimensional but preserves the spatial locality of the RTT measurements.

C. Network Architecture: Cross-Graph Design

This subsection explains our network architecture based on cross-graph learning, where two different GCN models for TMG $G_A(\mathcal{N}, \mathcal{E}_A)$ and DMG $G_B(\mathcal{N}, \mathcal{E}_B)$ are jointly trained by sharing their weights. As mentioned in Remark 1, we aim to preserve TMG and DMG's properties without catastrophic forgetting, and weight sharing is one representative technique to this end in the area of GNN (see, e.g., [34] and [35]). To this end, we use a two-layer GCN model for the two graphs, as illustrated in Fig. 4. Specifically, the first layer's vector representations, whose dimensions are set to h_1 , are a function of a weight matrix $W_1 \in \mathbb{R}^{2Q \times h_1}$, given as

$$H_A^{(1)}(W_1 | G_A) = \text{ReLU}(\hat{A} F_1 Q W_1), \quad (16)$$

$$H_B^{(1)}(W_1 | G_B) = \text{ReLU}(\hat{B} F_2 W_1). \quad (17)$$

Here, $\hat{A} \in \mathbb{R}^{N \times N}$ and $\hat{B} \in \mathbb{R}^{N \times N}$ are the normalized adjacency matrices, given as

$$\hat{A} = D_A^{-\frac{1}{2}} A D_A^{-\frac{1}{2}}, \quad \hat{B} = D_B^{-\frac{1}{2}} B D_B^{-\frac{1}{2}}, \quad (18)$$

²The half-length constraint on creating edges in DMG is based on the rule of thumbs, which allows us to stably train the proposed GNN architecture specified in the sequel.

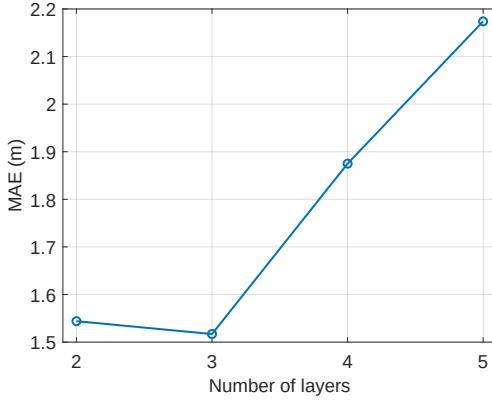


Fig. 5. MAE of MINGLE when more GCN layers are added. Experiment settings are specified in Sec. IV.

where $\mathbf{D}_A$ and $\mathbf{D}_B$ are diagonal matrices whose elements represent the sum of the corresponding row vectors in $\mathbf{A}$ and $\mathbf{B}$, respectively. We set $h_1 = 3000$ throughout the work. One observes that the input features $\mathbf{F}_1$ and $\mathbf{F}_2$ are separately fed into the GCN models for TMG and DMG, respectively. This mapping between the generated graphs and input features aligns well with the discussions in Remarks 1 and 2, whose optimality will be experimentally studied in Sec. IV. Besides, we multiply a *multi-layer perceptron* (MLP) weight matrix $\mathbf{Q} \in \mathbb{R}^{M \times 2Q}$ with $\mathbf{F}_1$. The resultant dimension then becomes equivalent to $\mathbf{F}_2$, facilitating the sharing of the common weight $\mathbf{W}_1$.

Next, the above first layer's outputs are recursively inputted to the second GCN layers, whose common weight matrix is denoted by $\mathbf{W}_2 \in \mathbb{R}^{h_1 \times 2}$. Then, the resultant embedding vectors, say $\mathbf{H}_A^{(2)} \in \mathbb{R}^{N \times 2}$ and $\mathbf{H}_B^{(2)} \in \mathbb{R}^{N \times 2}$, are functions of both $\mathbf{W}_1$ and $\mathbf{W}_2$, given as

$$\begin{aligned} \mathbf{H}_A^{(2)}(\mathbf{W}_1, \mathbf{W}_2 | \mathbf{G}_A) &= \hat{\mathbf{A}} \mathbf{H}_A^{(1)} \mathbf{W}_2 \\ &= \hat{\mathbf{A}} \left(\text{ReLU}(\hat{\mathbf{A}} \mathbf{F}_1 \mathbf{Q} \mathbf{W}_1) \right) \mathbf{W}_2, \end{aligned} \quad (19)$$

$$\begin{aligned} \mathbf{H}_B^{(2)}(\mathbf{W}_1, \mathbf{W}_2 | \mathbf{G}_B) &= \hat{\mathbf{B}} \mathbf{H}_B^{(1)} \mathbf{W}_2 \\ &= \hat{\mathbf{B}} \left(\text{ReLU}(\hat{\mathbf{B}} \mathbf{F}_2 \mathbf{W}_1) \right) \mathbf{W}_2. \end{aligned} \quad (20)$$

As a result, the RTT measurements at the n -th MP, say τ_n of (1), can be transformed to two vector representations embedded into 2D space, given as

$$\mathbf{a}_n = r_n(\mathbf{H}_A^{(2)}), \quad \mathbf{b}_n = r_n(\mathbf{H}_B^{(2)}), \quad (21)$$

where $r_n(\cdot)$ returns the n -th row vector in the matrix therein. We aim to train the weight matrices $\mathbf{W}_1$ and $\mathbf{W}_2$ to generate the final vector representations $\{\mathbf{a}_n\}$ and $\{\mathbf{b}_n\}$ that should be similar to the ground-truth locations $\{\mathbf{x}_n\}$ by minimizing the loss function introduced in the following subsection.

Remark 3 (Number of GCN Layers). The proposed MINGLE has a two-layer GCN architecture that is relatively shallow compared to the conventional deep CNN architecture. Fig. 5 shows that adding more layers does not significantly improve positioning accuracy but degrades it due to the vanishing gradient problem [36]. Several designs to train deep GCNs, such as residual connections, dense connections, and dilated

convolution, may provide better accuracy while costing higher computation complexity. It is interesting to optimize the accuracy-complexity trade-off, which will remain in our future work.

D. Loss Function Design

By concerning three conditions specified in (7), (8), and (9), we design a loss function, denoted by $\text{Loss}(\{\mathbf{a}_n\}, \{\mathbf{b}_n\})$, which comprises the two terms representing the MSE of the location estimates and their inter-distances regularized by the user mobility.

1) Mean Square Error: The MSE of the location estimates can be divided into two cases with and without ground-truth locations. First, consider a few MPs whose ground truth locations are given, say $\mathbf{x}_n$ with $n \in \mathcal{X}^c$. By inputting the vector representations of (21), we can define the following MSE-based loss function:

$$\text{MSE}_1(\{(\mathbf{a}_n, \mathbf{b}_n)\}_{n \in \mathcal{X}^c}) = \sum_{n \in \mathcal{X}^c} \|\mathbf{a}_n - \mathbf{x}_n\|^2 + \|\mathbf{b}_n - \mathbf{x}_n\|^2, \quad (22)$$

whose minimization leads to satisfying the first condition of (7).

Second, inspired by [14], we can use the CDA-based location estimates as their real-time labels. Despite the errors on such labels, it has been experimentally verified that decent positioning accuracy is achievable, even superior to the labels themselves. Denote $\mathbf{c}_n$ a CDA-based location estimate at the n -th MP. For brevity, the detailed algorithm to derive $\mathbf{c}_n$ is omitted but summarized in Appendix A. Given $\{\mathbf{c}_n\}$, we can compute an MSE-based loss function for the MPs without ground truths, given as

$$\text{MSE}_2(\{(\mathbf{a}_n, \mathbf{b}_n)\}_{n \in \mathcal{X}}) = \sum_{n \in \mathcal{X}} \|\mathbf{a}_n - \mathbf{c}_n\|^2 + \|\mathbf{b}_n - \mathbf{c}_n\|^2. \quad (23)$$

By equally weighting (22) and (23), we can obtain the overall MSE-based loss function, given as

$$\begin{aligned} \text{Loss}_{\text{MSE}}(\{\mathbf{a}_n, \mathbf{b}_n\}) &= \frac{1}{2} \text{MSE}_1(\{(\mathbf{a}_n, \mathbf{b}_n)\}_{n \in \mathcal{X}^c}) \\ &\quad + \frac{1}{2} \text{MSE}_2(\{(\mathbf{a}_n, \mathbf{b}_n)\}_{n \in \mathcal{X}}). \end{aligned} \quad (24)$$

2) Mobility Regularization: Recalling the assumption of the constant speed on the same SC in Assumption 1, we formulate the problem of minimizing the variance of the inter-distance between adjacent MPs, which is a surrogate for the constraints (8) and (9). Given the vector representation $\mathbf{b}_n$ in (21), we denote γ_n the inter-distance between the $(n-1)$ -th and n -th MPs' location estimates normalized by the speed variation factor $v(\ell)$ of (5) for $n \geq 2$, namely,

$$\gamma_n = \frac{\|\mathbf{b}_n - \mathbf{b}_{n-1}\|}{v(\ell^*)}, \quad \ell^* = \{\ell | n \in \{s(\ell), \dots, e(\ell)\}\}. \quad (25)$$

Then, the corresponding loss function is given as

$$\text{Loss}_{\text{MR}}(\{\mathbf{b}_n\}) = \text{var}(\{\gamma_n\}), \quad (26)$$

where $\text{var}(\cdot)$ represents the variance of the sequence therein.

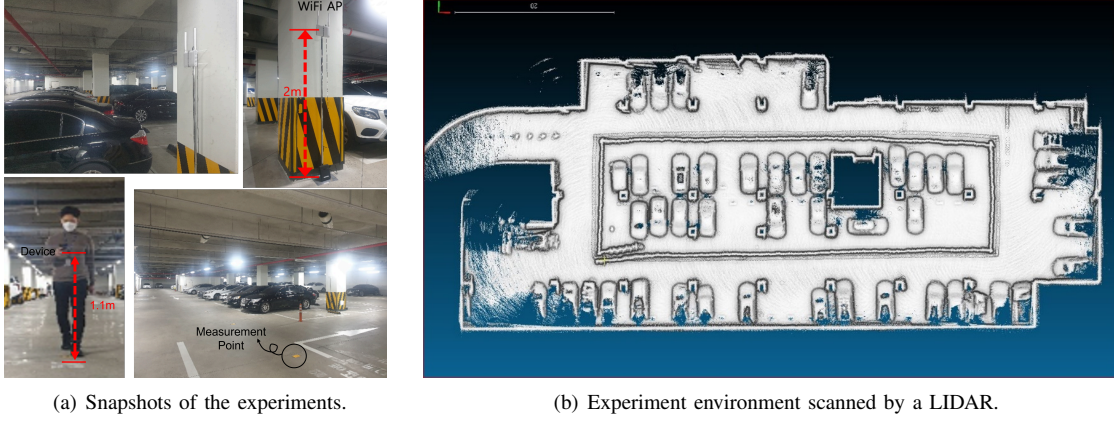


Fig. 6. Several photos of our experiment site and its floor plan scanned by a LIDAR. Parked cars' locations vary from day to day.

TABLE II
EMPIRICAL TIME COMPLEXITY ANALYSIS. EXPERIMENT SETTINGS ARE SPECIFIED IN SEC. IV.

Type #	1	2	3	4
Number of MPs	70	34	83	95
Average time span [s]	138.2	40.7	78.9	127.9
Average epochs	2913	1414	1110	1983
Average time per one MP [s]	1.97	1.20	0.95	1.35

By summing up the two loss functions (24) and (26), we can derive the overall loss function as

$$\text{Loss}(\{\mathbf{a}_n\}, \{\mathbf{b}_n\}) = \frac{1}{1 + \lambda} \text{Loss}_{\text{MSE}}(\{\mathbf{a}_n, \mathbf{b}_n\}) + \frac{\lambda}{1 + \lambda} \text{Loss}_{\text{MR}}(\{\mathbf{b}_n\}), \quad (27)$$

where $\lambda \geq 0$ is a weight parameter. After the convergence of the loss function (27), we use the second vector representation $\{\mathbf{b}_n\}$ in (21) as the location estimates, namely,

$$\hat{\mathbf{x}}_n = \mathbf{b}_n, \quad \forall n \in \mathcal{N}, \quad (28)$$

whose optimality has been extensively studied through field experiments and shown to be more accurate than the other vector representation $\{\mathbf{a}_n\}$ in Sec. IV-C.

Remark 4 (Transductive Learning). The existing DNN-based positioning algorithms require offline training using numerous measurement data samples collected in advance. On the other hand, the proposed MINGLE is designed based on transductive learning [37], such that training the GNN model and inferring the location estimates are simultaneously made without an offline training phase. In other words, MINGLE requires online measurements obtained when a user is on the current mobility path, making a practical choice in scenarios where prior measurements and their long-term statistics become invalid quickly over time.

Remark 5 (Time Complexity). MINGLE efficiently traces a user trajectory by localizing all MPs on the mobility path, yet causing a long time span, say around 1-2 minutes, as Table II summarizes. The current design of MINGLE may be unsuitable to a real-time response. Considering that the average time per one MP is around 1 (sec), it is interesting

to modify MINGLE for real-time positioning by sequentially estimating each MP, which remains as future work.

IV. FIELD EXPERIMENTS

This section has validated the proposed MINGLE through extensive field experiments. The experiment site and settings are explained first. Then, we compare MINGLE with several benchmarks from various perspectives.

A. Experiment Settings

We conducted field experiments in an underground parking lot at the Korea Railroad Research Institute, Uiwang, Korea. We installed 10 WiFi APs at the height of 2 (m), which is the maximum number of WiFi APs from which one smartphone simultaneously collects RTTs. The user holds a smartphone at the height of 1.1 (m). Two kinds of smartphones (Google Pixel 2XL and Pixel 3a) are used to measure RTTs from APs. As shown in Fig. 6, several obstacles exist in the concerned site, such as cars and pillars, which frequently block the LoS path between the smartphone and APs. For training and inference, the hardware specifications are an AMD Ryzen 9 7950X3D for the CPU and a RTX 2070 Super for the GPU.

Four types of user mobility trajectories are considered, as represented in Fig. 7. Except for the last type, every trajectory follows a rectangular mobility pattern with 4 SCs, but the number of MPs therein are different, say 70, 34, and 83 MPs for Types 1, 2, and 3, respectively. Besides, the distance between adjacent MPs on Type-1 and Type-2 patterns is constant, while the Type-3 pattern has different inter-distance depending on vertical and horizontal paths to reflect different moving speeds. Contrary to the other types, the Type-4 trajectory comprises 8 SCs (two SCs are overlapped) with 95 MPs. Since only a limited time is allowed for the experiment per day, we conduct 10 experiments for 3 days. Table III gives a detailed explanation of all experiments. It is worth mentioning that the number and locations of parked cars differed for each day, resulting in a non-stationary RTT distribution, as summarized in Appendix B. In particular, RTT on Day 3 was more corrupted than those on Days 1 and 2.

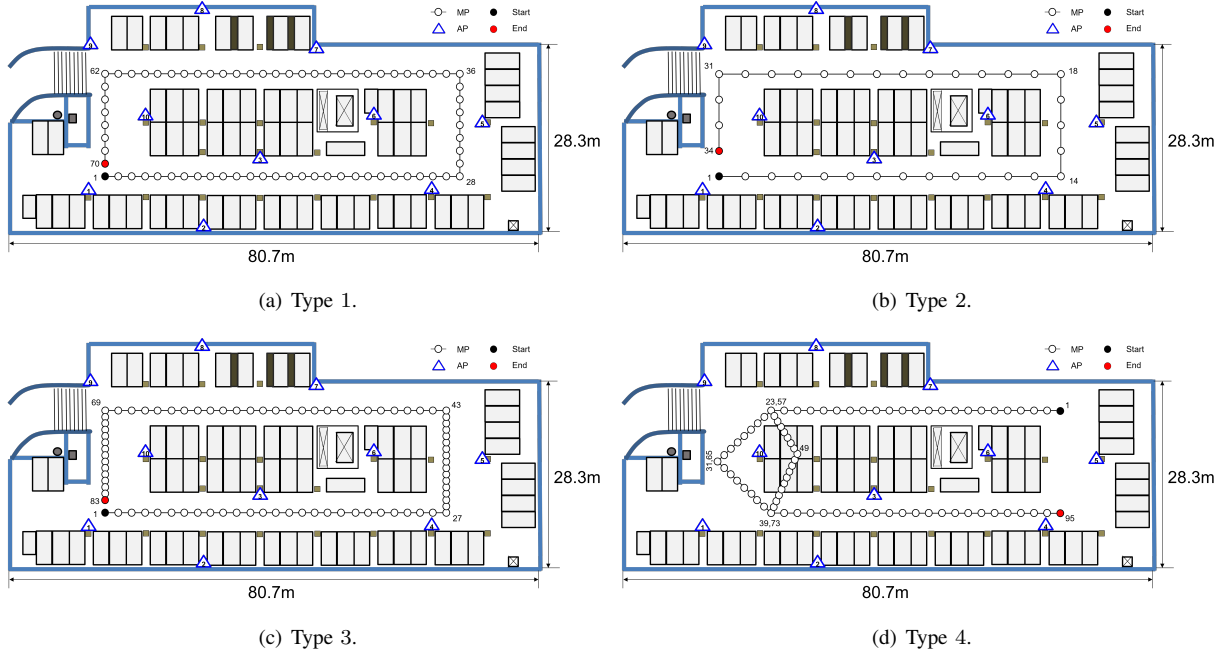


Fig. 7. Four types of user trajectories. The location of APs are represented by blue triangles, and the circles represent MPs. The detailed information of each trajectory is summarized in Table III.

As Remark 4 mentioned, the proposed MINGLE with two GCN layers is trained anew for each experiment using the corresponding RTT measurements only. We exclusively divide every MP's RTT into training and validating sets with a ratio of 8:2. The nodes in the training set are only included when computing the loss function of (24) for training, while the convergence of the model is verified by the loss function computed by the nodes in the validation set. The final result after the convergence becomes the sequence of position estimates. The number of training epochs is up to 6000, which can be early terminated unless the validation loss is reduced within 200 epochs. We repeated the training process five times to obtain more reliable results by randomizing validation sets, initial weights, and seed numbers. The resultant sequence of position estimates can be derived by computing their median. We set the parameter determining the temporal adjacency ϵ specified in (10) as 2. The weight of the mobility regularization term λ specified in (27) is set to 3 for Days 1 and 2, and 30 for Day 3, whose optimality will be discussed in Sec. IV-E. For a semi-supervising setting, we assume that the ground-truth locations are given when the user's moving direction is changed, namely, $\alpha_n = \beta_n$ ³ for all $n \in \mathcal{N}$.

We have conducted additional experiments in a new site (Geumnamno4-ga subway station, Gwangju, South Korea) for further verification. The experiment setting and results are summarized in Appendix D.

B. Performance Comparison with Benchmarks

This subsection compares MINGLE's positioning accuracy with several benchmarks introduced below.

³To make our semi-supervised learning problem nontrivial, the starting MP is not included in the set of the ground-truth locations. The result when adding the starting MP is summarized in Appendix C.

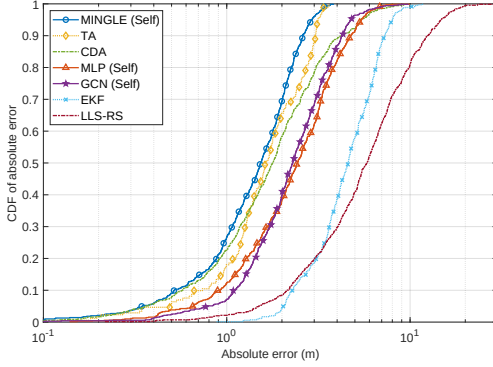
TABLE III
ENVIRONMENT DESCRIPTION.

Experiments #	1	2	3	4	5	6	7	8	9	10
Experiment day	Day 1	Day 2			Day 3					
Device	Pixel 2XL				Pixel 3a					
Path shape	Type 1	Type 2			Type 3			Type 4		
Number of node	70	34	34	34	83	83	83	95	95	95
Number of SCs	4	4	4	4	4	4	4	8	8	8

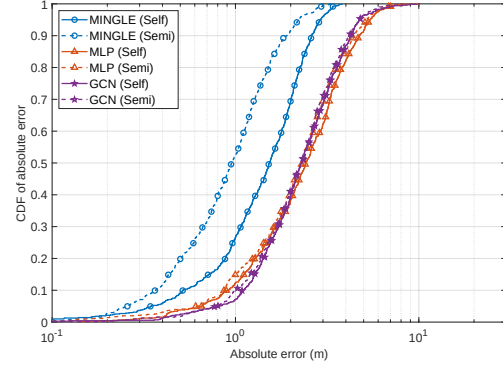
- **MLP**: A DNN-based technique without graphs, which replaces all GCN layers with MLP layers. It is experimentally verified that the upper MLP's output, whose input is $\mathbf{F}_1$ of (19), provides a better positioning accuracy than those of the lower MLP, whose input is $\mathbf{F}_2$ of (20). Eventually, we use the upper MLP's outputs as location estimates. The other architecture and parameters are equivalent to MINGLE.
- **Linear least square through reference selection (LLS-RS)** [33]: One representative multilateration algorithm assuming that all RTTs are measured under LoS conditions.
- **CDA** [14]: Our previous work to overcome NLoS propagation by obtaining multiple PELs through different AP combinations. MINGLE uses the CDA-based location estimates as real-time labels for a self-supervised approach. The detailed process of CDA is provided in Appendix A.
- **TA** [15]: Another previous work aligning IMU-based user trajectory into WiFi RTT measurements. Noting that TA is based on the assumption of a constant moving speed, it can be applicable only when the distance between adjacent MPs is equivalent, as in the case of Type 1 and Type 2 in Fig. 7.
- **EKF** [11]: A representative fusing technique of WiFi positioning and PDR. The parameters determining the

TABLE IV
SUMMARY OF EXPERIMENTS RESULTS IN SEC. IV-B (m). THE METHOD WITH THE BEST POSITIONING ACCURACY IS REPRESENTED BY BOLD.

RTT distribution		Day 1	Day 2				Day 3					Average	50th	75th	95th	RMSE
Path shape		Type 1	Type 2			Type 3			Type 4							
Learning	Method	1	2	3	4	5	6	7	8	9	10					
Self-supervised	MINGLE	1.197	1.361	1.429	1.188	1.366	1.395	1.554	1.957	1.899	1.749	1.510	1.530	2.109	2.903	1.755
	MLP	2.067	2.014	2.169	2.117	2.646	3.052	2.598	2.637	2.923	2.888	2.511	2.424	3.513	5.423	3.044
	LLS-RS	5.525	6.250	6.403	6.771	7.130	7.777	7.160	6.034	6.343	6.587	6.598	5.803	8.861	14.663	7.853
	CDA	1.880	1.800	1.640	1.697	2.510	2.430	2.175	2.150	2.158	2.138	2.058	1.776	2.819	5.366	2.663
	TA	1.705	1.328	1.724	2.466	—	—	—	—	—	—	1.806	1.620	2.515	3.139	1.977
	EKF	5.243	3.616	3.716	3.449	5.077	5.539	4.923	4.582	4.708	5.224	4.608	4.580	6.190	8.012	5.147
	GCN	1.648	2.993	3.110	2.974	2.292	2.515	2.251	2.460	2.708	2.532	2.548	2.313	3.225	4.765	2.770
Semi-supervised	MINGLE	1.060	1.214	1.303	1.104	0.802	0.932	0.926	1.154	1.244	1.028	1.077	0.951	1.381	2.196	1.217
	MLP	2.085	2.135	2.544	2.033	2.555	2.705	2.537	2.493	2.616	2.534	2.424	2.316	3.296	5.151	2.871
	GCN	1.747	2.991	3.148	3.001	2.301	2.620	2.302	2.602	2.752	2.656	2.612	2.292	3.290	4.768	2.866



(a) Self-supervised learning.



(b) Semi-supervised learning.

Fig. 8. CDFs of absolute errors for MINGLE and benchmarks.

weight of WiFi and PDR results are experimentally optimized.

- GCN [2]: Motivated by [2], we consider a GCN-based benchmark, shortly called GCN, as follows. All MPs become the corresponding graph's nodes, and their connections are made when their inter-distances are below the predetermined threshold. To this end, we assume that all ground-truth inter-distances are known in advance⁴.

For a fair comparison, we compute various performance metrics, including the 50th, 75th, and 95th percentiles of absolute errors, *mean absolute error* (MAE), and *root mean square error* (RMSE). All results are summarized in Table IV. In the following, we highlight MINGLE's superiority over benchmarks by dividing the cases into self-supervised learning (with no labels) and semi-supervised learning (with a few labels).

1) *Self-Supervised Learning*: First, Fig. 8(a) shows the *cumulative distribution function* (CDF) of the absolute error when no ground-truth location is given for training. It is shown that MINGLE outperforms the others except for TA. Recalling that TA cannot be applicable for Type-3 and Type-4 trajectories due to the assumption of a constant step length, MINGLE is said to be the most effective algorithm from the perspectives of accuracy and applicability.

⁴It is worth noting that the original method in [2] considers all APs and MPs as nodes, which is efficient when their numbers are sufficient and balanced. Our scenario with 10 APs, which is much smaller than the number of MPs, causes the unbalance between two types of nodes, failing to achieve a reliable positioning result. We thus use only MPs as nodes.

One noticeable point is the comparison to CDA's results, whose location estimates are used for real-time labels of MINGLE. As observed in Fig. 9, MINGLE tends to smooth nearby location estimates by leveraging two mobility-induced graphs. On the other hand, CDA-based location estimates are derived using the corresponding RTT measurements only, prone to severe NLoS conditions. Consequently, MINGLE is effective in revising CDA's position estimates deviating from the ground truth, say the 95th percentile of MINGLE being 2.903 (m) while CDA counterpart being 5.366 (m).

2) *Semi-Supervised Learning*: Recalling that in the case of semi-supervised learning, a few ground-truth locations are given for the MPs whose heading directions are changed, e.g., $\beta_n = 1$. To verify the effect of such ground-truth labels, we compare the results of self- and semi-supervised learning in Fig. 8(b), showing a significant performance enhancement. As shown in Fig. 10, MINGLE can utilize the given ground-truth locations to find the other locations, effective for Type-3 and Type-4 trajectories. We also compare the results of GCN [6] as a benchmark, revealing no performance enhancement. Eventually, MINGLE is said to be better at utilizing the ground-truth locations than the benchmark.

Next, we aim to show the effect of number of ground-truth labels on the positioning performance. To this end, we uniformly pick 5-20% MPs on the trajectories as ground-truth labels, and measure the MAE of the semi-supervised version of MINGLE in Fig. 11. It is observed that a small portion of labels makes the MAE significantly reduced, say from 1.282 (m) to 1.003 (m) when 5% labels are given. The gain of

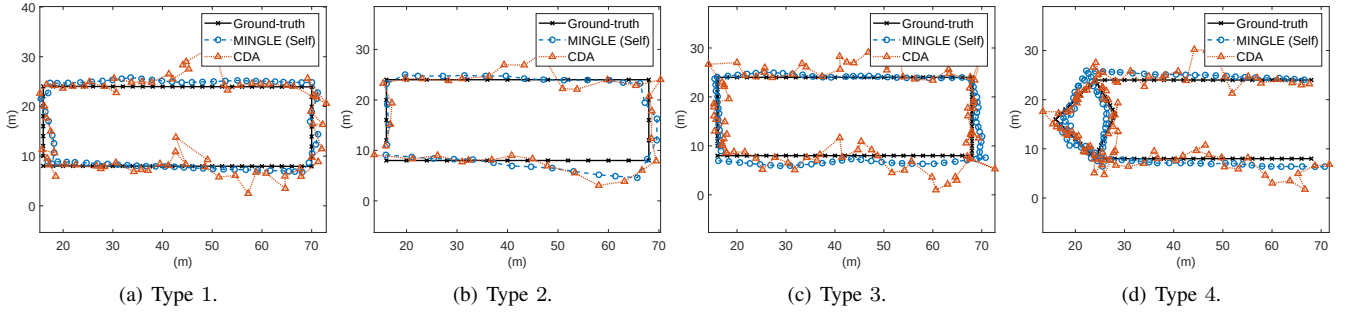


Fig. 9. Estimated user trajectories of MINGLE and CDA when no ground-truth location is given (self-supervised learning).

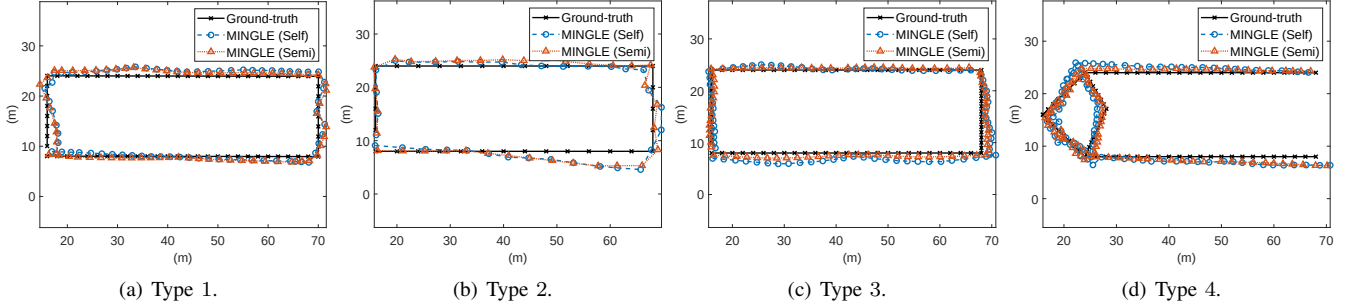


Fig. 10. Estimated user trajectories of MINGLE when a few ground-truth location is given (semi-supervised learning). We depict the result of self-supervised version of MINGLE for a fair comparison.

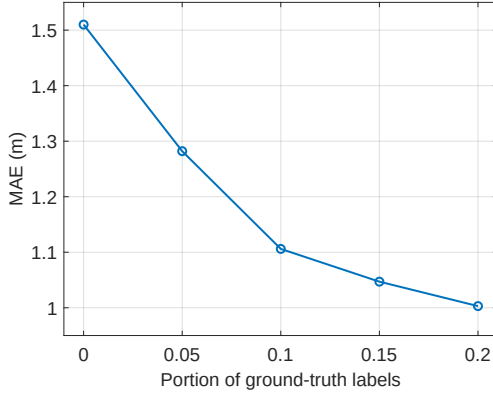


Fig. 11. MAE vs. the portion of ground-truth labels.

number of the label decreases as the portion increases, and the resultant MAE tends to converge with no more performance enhancement.

Last, it is worth highlighting that there is a significant gap between MINGLE and MLP for both self- and semi-supervised learning cases. Our positioning problem, where only a few RTT data samples are given for the current mobility path, is inappropriate for MLP since it relies on a large number of data samples. By comparing MINGLE and MLP, we confirm that leveraging the two mobility-induced graphs can overcome the shortage of data samples, which is a crucial contribution to the work.

C. Cross-Graph Learning

As recalled in Sec. III-C, we develop two input features, say F_1 and F_2 , which are inputted to TMG- and DMG-based

GCNs with sharing their weights. This subsection aims to confirm the optimality of the proposed cross-graph learning. To this end, we consider two strategies for training the network.

- **Standalone Learning:** The first strategy is to train a standalone network for each individual graph. Noting that we have two inputs and two graphs, allowing us to make 4 combinations for standalone training strategy, namely, F_1 -TMG, F_2 -TMG, F_1 -DMG, and F_2 -DMG.
- **Cross-Graph Learning:** The second strategy is a cross-graph learning, making 4 options, namely, (F_1 -TMG, F_1 -DMG), (F_1 -TMG, F_2 -DMG), (F_2 -TMG, F_1 -DMG), and (F_2 -TMG, F_2 -DMG).

We use the settings specified in Section IV-A except for (F_2 -TMG, F_2 -DMG). Specifically, we set $\lambda = 3$ instead of our primary setting with $\lambda = 30$ for stable training. All results of this study are summarized in Table V. Several interesting observations are made. First, every standalone approach performs worse than its cross-graph counterpart, justifying the importance of cross-graph learning. Second, the primary feature-graph matching, referred to as (F_1 -TMG, F_2 -DMG), achieves the best positioning accuracy in semi-supervised learning, while the reverse matching, referred to as (F_2 -TMG, F_1 -DMG), provides the best performance in self-supervised learning. Nevertheless, the gap between the two is marginal, allowing us to conclude that feature-graph mapping is not a critical factor, given the effectiveness of the proposed cross-graph learning.

To understand each graph's contribution in detail, we illustrate the estimated Type-4 trajectories of our cross-graph learning in Fig. 12, namely, the outputs of TMG and DMG,

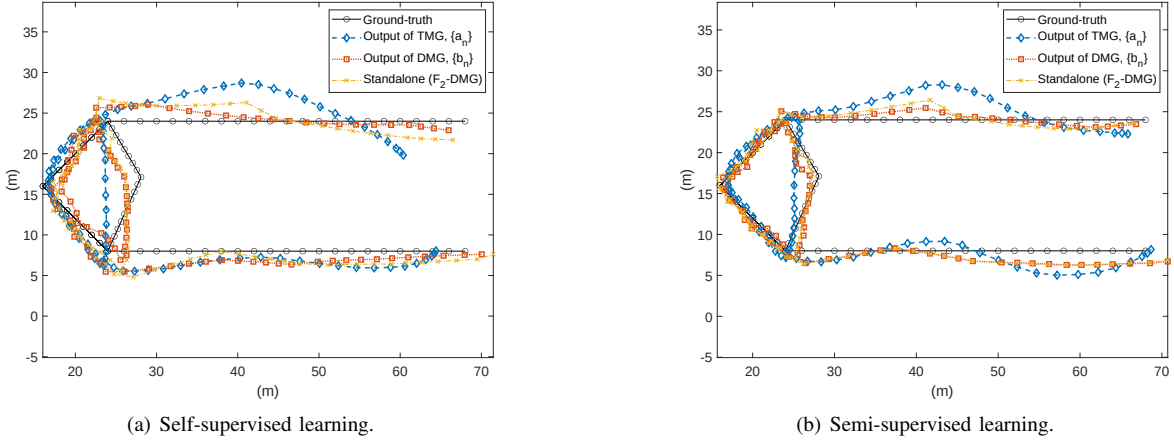


Fig. 12. Estimated user Type-4 trajectories. For cross-graph learning, the outputs of TMG and DMG, say $\{a_n\}$ and $\{b_n\}$ in proposed MINGLE (F_1 -TMG, F_2 -DMG), respectively, are illustrated. For standalone learning, the output of F_2 -DMG is illustrated.

TABLE V
FEATURE-GRAPH COMBINATION (m). THE BEST COMBINATION IS HIGHLIGHTED IN BOLD.

Experiment day				Day 1		Day 2		Day 3						Average	50th	75th	95th	RMSE
Path shape				Type 1	Type 2		Type 3		Type 4									
Learning	Method	Graph 1	Graph 2	1	2	3	4	5	6	7	8	9	10					
Self-supervised	Stand-alone	F ₁ -TMG	-	2.833	2.316	2.273	2.410	2.474	2.682	2.274	2.494	2.740	2.447	2.494	2.414	3.253	4.565	2.789
		F ₂ -TMG	-	1.911	1.729	1.728	2.004	2.492	2.586	2.472	2.392	2.542	2.443	2.230	2.111	3.099	4.792	2.674
		-	F ₁ -DMG	4.741	5.666	5.101	4.768	5.050	4.799	4.891	3.282	3.140	3.073	4.451	3.851	5.960	9.766	5.179
		-	F ₂ -DMG	1.216	1.368	1.455	1.297	1.507	1.651	1.699	1.858	2.119	1.736	1.591	1.590	2.164	3.054	1.828
	Cross-graph	F ₁ -TMG	F ₁ -DMG	1.591	2.322	2.579	2.300	2.636	2.613	2.418	2.733	2.838	2.792	2.482	2.827	4.257	10.369	4.641
		F ₁ -TMG	F ₂ -DMG	1.197	1.361	1.429	1.188	1.366	1.395	1.554	1.957	1.899	1.749	1.510	1.530	2.109	2.903	1.755
		F ₂ -TMG	F ₁ -DMG	1.195	1.336	1.488	1.209	1.426	1.317	1.646	1.911	1.929	1.586	1.504	1.522	2.050	2.699	1.709
		F ₂ -TMG	F ₂ -DMG	3.424	1.490	1.517	1.363	1.435	1.422	1.587	1.838	1.979	1.588	1.764	1.693	2.264	3.086	2.208
Semi-supervised	Stand-alone	F ₁ -TMG	-	1.698	1.585	1.710	1.604	2.903	2.507	2.228	2.238	2.164	2.114	2.075	1.827	3.093	4.648	2.588
		F ₂ -TMG	-	1.806	1.804	1.844	1.910	2.247	2.415	2.329	1.871	1.910	1.883	2.002	1.679	2.794	4.773	2.408
		-	F ₁ -DMG	7.229	5.226	5.302	5.026	4.034	3.475	3.130	3.080	3.188	2.502	4.219	3.000	4.980	10.730	4.977
		-	F ₂ -DMG	1.077	1.241	1.639	1.189	1.289	1.228	1.246	1.269	1.340	1.133	1.265	1.069	1.696	2.654	1.454
	Cross-graph	F ₁ -TMG	F ₁ -DMG	1.756	2.389	2.268	2.146	2.459	2.425	2.179	2.041	1.912	1.795	2.137	2.282	3.904	10.897	4.665
		F ₁ -TMG	F ₂ -DMG	1.060	1.214	1.303	1.104	0.802	0.932	0.926	1.154	1.244	1.028	1.077	0.951	1.381	2.196	1.217
		F ₂ -TMG	F ₁ -DMG	1.065	1.233	1.334	1.111	0.898	1.079	0.935	1.245	1.264	1.023	1.119	0.974	1.406	2.325	1.277
		F ₂ -TMG	F ₂ -DMG	1.426	1.447	1.340	1.229	0.978	1.070	1.148	1.272	1.300	1.105	1.232	1.072	1.641	2.566	1.397

TABLE VI
MAE (m) WHEN FULL-, HALF-, AND QUARTER-TRAJECTORIES ARE USED.

Trajectory	1	2	3	4	5	6	7	8	9	10	Average	50th	75th	95th	RMSE
Full	1.197	1.373	1.445	1.211	1.366	1.395	1.554	1.957	1.899	1.749	1.515	1.530	2.109	2.903	1.755
Half	1.111	1.137	1.279	1.122	2.227	2.177	2.045	2.063	2.752	2.665	1.858	1.770	2.937	4.639	2.439
Quarter	1.405	1.963	2.216	2.046	3.175	2.634	2.426	2.192	2.492	2.567	2.312	1.996	3.061	5.436	2.925

say $\{a_n\}$ and $\{b_n\}$ specified in (21), respectively. Besides, we add the result of the standalone learning for F_2 -DMG. It is shown that for cross-graph approach, the output of DMG plays a more crucial role in finding the location than that of TMG. It is thus reasonable to use the DMG's output as the final location estimate, as specified in (28). Next, from comparing the standalone approach of F_2 -DMG observing the misalignment between different SCs, the cross-graph learning approach seamlessly connects all nodes and preserves each SC's straight transition. In summary, F_2 -DMG dominantly contributes to achieving accurate positioning, but F_1 -TMG helps improve the connectivity between SCs through cross-graph learning.

D. Effect of Trajectory Length

To show the effect of the trajectory length on MINGLE's positioning performance, we divide each trajectory into 2 half-trajectories and 4 quarter-trajectories, and apply MINGLE to

them. The resultant MAEs of all segmented trajectories are summarized in Table VI. It is shown that the case with an entire trajectory performs better than the other, aligned with our intuition of the number of training data samples. To explain, as the concerned trajectory becomes reduced, similar results among three different datasets are observed in Experiments 1 to 4, while a significant performance degradation is observed in Experiments 5 to 10. This decline is attributed to noisy RTT measurements at Day 3 (See Appendix B). In other words, a longer trajectory is more beneficial to compensate for noisy measurements of WiFi RTT.

E. Effect of Mobility Regularization Term

This subsection aims to check the effect of the mobility regularization term $\text{Loss}_{\text{MR}}(\{b_n\})$ of (26) by changing its weight factor λ from 0 to 30. Fig. 13 depicts the MAE for each experiment day in terms of λ for self- and semi-supervised learning. Noting that there is no mobility regularization when

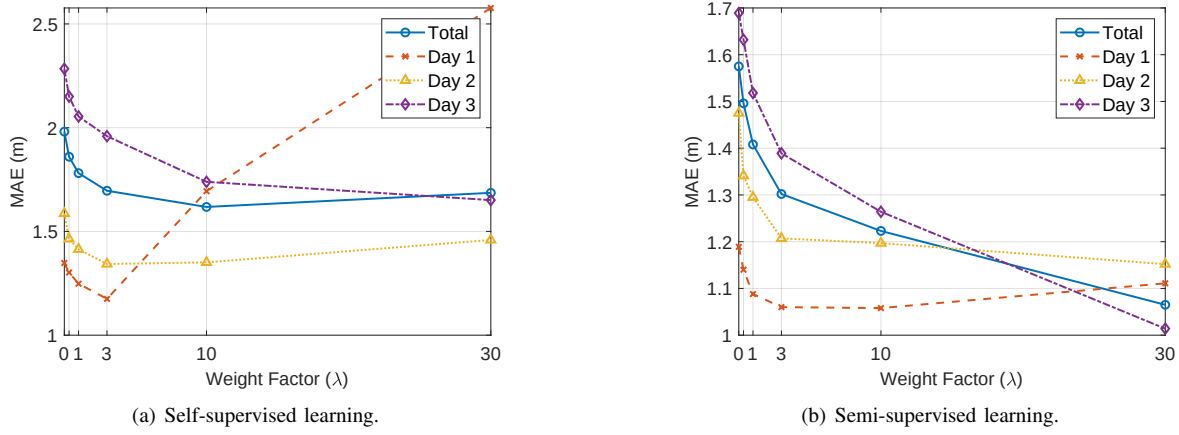


Fig. 13. Effect of mobility regularization term's weight factor λ .

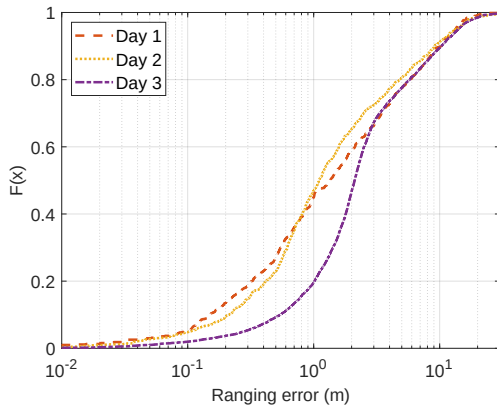


Fig. 14. CDF of RTT ranging error. The number of cars parked in the experiment site and their locations cause non-stationary RTT distribution over days, resulting in different positioning performance.

$\lambda = 0$, the performance gain can be defined as the MAE gap from the minimum to the case when $\lambda = 0$. Several interesting observations are made. First, in the case of self-supervised learning, the optimal λ differently exists depending on the experiment day, say $\lambda = 3$ for Days 1 and 2 while $\lambda = 30$ for Day 3, justifying the setting of λ in Sec. IV-A. In the case of semi-supervised learning, on the other hand, the MAE is a monotone decrease of λ in a concerned range. Second, the gain of the mobility regularization term varies depending on the experiment day. One shows that the MAE of Day 3 is the most significantly degraded, with the most noisy RTT measurements. Consequently, we confirm the importance of the mobility regularization term to enable self- and semi-supervised learning, and its effect becomes more dominant if a few reliable labels are added or raw RTT measurements are severely corrupted.

V. CONCLUDING REMARKS

This paper has proposed a novel positioning technique called MINGLE, which is designed based on GNN using real-time graphs created by observing the user's mobility patterns. MINGLE has three key features to address several

limitations of the existing works. First, the created mobility-induced graphs, say TMG and DMG, can be obtainable using a smartphone's low-resolution built-in IMU since they only require the IMU measurements' binary quantization values. Second, MINGLE enables us to seamlessly integrate WiFi positioning and user mobility without concern about their significance since WiFi RTT measurements and mobility information contribute to a different part of the proposed architecture. Third, CDA-based real-time labels and the mobility regularization term help design MINGLE with no or a few ground-truth labels. As a result, MINGLE is considered an effective positioning technique in practice, which is experimentally verified by showing more accurate positioning results than benchmarks.

The current work can be extended in several directions. This work considers a single-user scenario where mobility-induced graphs are created from a single user's mobility. The extension to a scenario with multiple users allows us to generate more graphs with different nodes and edges, making the problem more interesting. Next, MINGLE is designed based on the assumption that a sufficient number of positioning anchors is given, which is sometimes challenging when only WiFi APs are considered. On the other hand, by securing sufficient positioning anchors, along with other radio technology, such as UWB and Bluetooth, it is possible to make MINGLE more effective. Last, applying MINGLE to vehicle-to-everything scenarios for integrated sensing and communication will be another promising direction.

APPENDIX

A. Combinatorial Data Augmentation

To create a self-supervised label $\mathbf{c}_n$, we utilize the CDA method described in [14]. From the set of APs $\mathcal{M}$, select K out of M APs, and consider all possible combinations grouped as $\{\mathcal{M}_q\}_{q=1}^Q$, where $Q = \binom{M}{K}$. For a subset $\mathcal{M}_q \subset \mathcal{M}$, we define two error metrics $u_n^{(q)}$ and $v_n^{(q)}$ using RTT $\tau_n^{(m)}$ in (1)

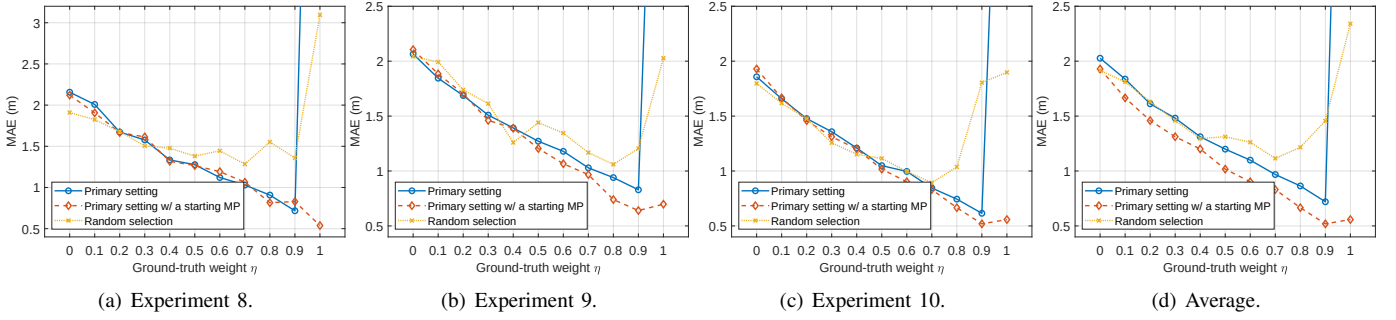


Fig. 15. The MAEs of Type-4 paths under different strategies of selecting ground-truth locations when the WMSE loss function of (33) is used instead of the MSE loss function of (24).

TABLE VII
POSITIONING ACCURACY OF TYPE-3 PATHS WHEN THE NUMBER OF MPs IS REDUCED (m).

	Type-1	Type-2	Type-3		
Number of MPs	70	34	83	68	34
MAE (m)	1.197	1.343	1.438	2.412	2.765

and PEL $\mathbf{y}^{(q)}$ in (14) as

$$u_n^{(q)} = \sum_{m \in \mathcal{M}_q} \left| \mathbf{z}^{(m)} - \mathbf{y}^{(q)} - \frac{c \cdot \tau_n^{(m)}}{2} \right|,$$

$$v_n^{(q)} = \sum_{m \in \mathcal{M}_q} \tau_n^{(m)}.$$
(29)

Next, we define a set of PELs as $\mathcal{Z}_n = \{\mathbf{y}^{(q)}\}_{q=1}^Q \in \mathbb{R}^{Q \times 2}$. Based on this set of PELs, the author of [14] proposed two filtered sets, denoted by $\mathcal{Z}_n^{(1)}$ and $\mathcal{Z}_n^{(2)}$, using the above error metrics as

$$\mathcal{Z}_n^{(1)} = \{\mathbf{y}^{(q)} \in \mathcal{Z}_n \mid u_n^{(q)} \leq u_n^{(q_1)}\},$$

$$\mathcal{Z}_n^{(2)} = \{\mathbf{y}^{(q)} \in \mathcal{Z}_n^{(1)} \mid v_n^{(q)} \leq v_n^{(q_2)}\},$$
(30)

where q_1 and q_2 represent the indices of PELs with the Q_1 -th and Q_2 -th lowest error among $\mathcal{Z}_n$ and $\mathcal{Z}_n^{(1)}$, respectively. Finally, the estimated position $\mathbf{c}_n$ is calculated as the median of $\mathcal{Z}_n^{(2)}$:

$$\mathbf{c}_n = \text{Median}(\mathcal{Z}_n^{(2)}).$$
(31)

In our experiments, we set $K = 3$, $Q_1 = 37$, and $Q_2 = 12$ for Experiments 1 to 4, and $K = 3$, $Q_1 = 40$, and $Q_2 = 20$ for Experiments 5 to 10, respectively. These parameters were optimized using numerical methods.

B. RTT Error Distribution

In our experimental settings, different scattering environments and smartphone devices can influence the RTT error distribution. We define the ranging error as the absolute error as:

$$e_n^{(m)} = \left| \mathbf{z}^{(m)} - \mathbf{x}_n - \frac{c \cdot \tau_n^{(m)}}{2} \right|.$$
(32)

Here, c represents the speed of light. We gather these errors into a set, denoted as $\{e_n^{(m)}\}$. The CDF of ranging error is

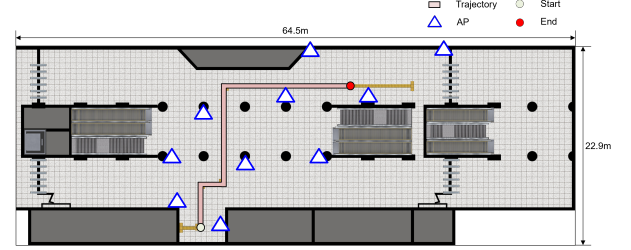


Fig. 16. Floor plan and user mobility path of the experiment site in Appendix D (Geumnamno4-ga subway station, Gwangju, South Korea).

visualized in Fig. 14. It reveals that the average ranging error on Day 3 is larger compared to those on Days 1 and 2.

Next, we aim to investigate the effect of each experiment day's RTT measurements by comparing Types 1, 2, and 3, whose trajectories are equivalent but measured in different days, say Days 1, 2, and 3, respectively. The number of MPs of the Type-3 path is reduced to 68 and 34 for fair comparisons to Type-1 and Type-2 paths, whose numbers of MPs are 70 and 34, respectively. As summarized in Table VII, Type-3's MAE is higher than those of Type-1 and Type-2 counterparts due to the negative effect of noisy RTT measurements of Day 3. On the other hand, Type-3's MAE is significantly reduced as MPs increase, confirming the effectiveness of the proposed MINGLE by leveraging two mobility-induced graphs, say TMG and DMG, independent of RTT measurements.

C. Comparison between Different Strategies to Select Ground-truth Locations

This subsection investigates the effect of selecting ground-truth locations by comparing the following three strategies: the primary setting in Sec. IV-A (i.e., $\forall n \in \mathcal{N}$ satisfying $\alpha_n = \beta_n$), the primary setting with a starting MP, and random selection. To this end, the MSE loss function of (24) is modified to the *weighted MSE* (WMSE) function, given as

$$\text{Loss}_{\text{WMSE}}(\{\mathbf{a}_n, \mathbf{b}_n\}) = \eta \text{MSE}_1(\{\{\mathbf{a}_n, \mathbf{b}_n\}\}_{n \in \mathcal{X}^c}) + (1 - \eta) \text{MSE}_2(\{\{\mathbf{a}_n, \mathbf{b}_n\}\}_{n \in \mathcal{X}}),$$
(33)

where $0 \leq \eta \leq 1$ represents the weight of nodes whose ground-truth location is given. The WMSE loss function is reduced to (24) when $\eta = \frac{1}{2}$. Fig. 15 illustrates the MAEs under those strategies when η changes from 0 to 1. Two

TABLE VIII

SUMMARY OF SUBWAY STATION EXPERIMENTS RESULTS (m). THE METHOD WITH THE BEST POSITIONING ACCURACY IS REPRESENTED BY BOLD.

Learning	Experience #	1	2	3	4	5	6	7	8	9	Average	50th	75th	95th	RMSE
	Number of MPs	57	53	55	53	54	54	57	61	65					
Self-supervised	MINGLE	1.778	1.500	1.383	1.269	1.553	1.504	1.584	1.403	1.346	1.480	1.517	1.865	2.459	1.605
	MLP	2.609	2.525	1.910	1.789	1.766	1.514	2.028	1.773	3.809	2.191	1.901	2.747	4.470	2.495
	LLS-RS	4.369	4.178	4.123	4.336	4.154	3.977	3.732	4.421	3.761	4.117	3.610	5.465	8.611	4.700
	CDA	2.125	1.829	1.830	1.795	1.963	1.778	1.967	1.797	1.849	1.881	1.666	2.437	3.865	2.148
	TA	2.854	2.754	2.310	2.638	2.805	4.203	2.366	3.086	2.172	2.799	2.759	3.854	5.044	3.111
	EKF	3.818	3.488	3.302	3.859	3.592	3.259	2.747	3.810	3.063	3.438	2.961	4.212	6.231	3.670
	GCN	1.901	1.705	1.566	1.520	1.733	1.727	1.756	1.564	1.709	1.687	1.687	2.345	3.037	1.899
Semi-supervised	MINGLE	1.305	1.014	1.078	0.952	0.959	1.116	1.062	0.864	0.890	1.027	0.905	1.367	2.278	1.187
	MLP	2.006	2.009	1.897	1.744	2.021	1.731	2.235	1.618	2.005	1.918	1.664	2.617	3.913	2.183
	GCN	1.921	1.688	1.566	1.500	1.773	1.704	1.781	1.596	1.719	1.694	1.674	2.387	3.053	1.903

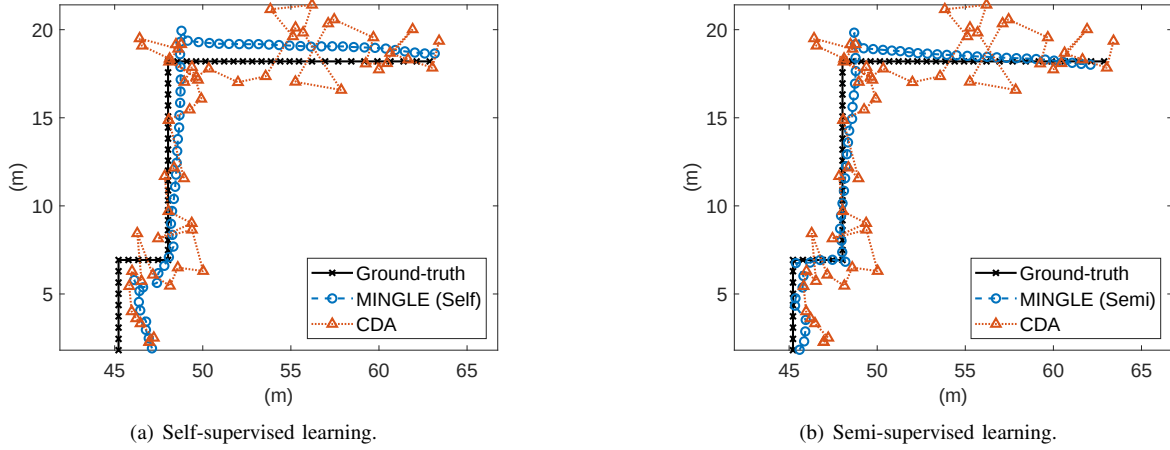


Fig. 17. Estimated user trajectories of MINGLE for self- and semi-supervised learning. The result of CDA is depicted for comparison. For the case of semi-supervised learning, the ground-truth locations are given at each SC's end.

key observations are made. First, the second strategy, whose positioning problem is trivially solvable by uniformly dividing the line segments between adjacent ground-truth locations, outperforms the others by fully weighting the nodes with ground-truth locations (i.e., $\eta = 1$), achieving the MAE near 0.5 (m). Second, with $\eta = \frac{1}{2}$, the MAE difference between the first and the third strategies is insignificant since MINGLE does not rely on the context of the ground-truth labels. It could be possible to optimize η depending on ground-truth selection if an in-depth site survey is available in advance, which is yet challenging in practice.

D. Additional Field Experiment

To confirm the proposed MINGLE's effectiveness in diverse environments, we have conducted additional experiments at a new site (Geumnamno4-ga subway station, Gwangju, South Korea). The floor plan of the experiment site is illustrated in Fig. 16. We set $\lambda = 3$, and all other experiment settings are equivalent to those in Sec. IV, unless specified otherwise. We conducted 9 experiments for one hour following the equivalent moving path, whose number of MPs was slightly different due to the user's random movement. We summarize all results in Table VIII. Fig. 17 showcases the proposed MINGLE, which outperforms benchmarks for both self- and semi-supervised learning, as in the cases in Sec. IV. As a result, we confirm its effectiveness in a relatively open place with fewer stationary obstacles but several pedestrians.

REFERENCES

- [1] C. Yang and H.-R. Shao, "Wi-Fi-based indoor positioning," *IEEE Commun. Mag.*, vol. 53, no. 3, pp. 150–157, 2015.
- [2] Y. Sun, Q. Xie, G. Pan, S. Zhang, and S. Xu, "A novel gcnn based indoor localization system with multiple access points," in *Proc. Int. Wireless Commun. Mobile Comput. (IWCMC)*, 2021, pp. 9–14.
- [3] X. Kang, X. Liang, and Q. Liang, "Indoor localization algorithm based on a high-order graph neural network," *Sensors*, vol. 23, no. 19, p. 8221, 2023.
- [4] X. Luo and N. Meratnia, "A geometric deep learning framework for accurate indoor localization," in *Proc. IEEE 12th Int. Conf. Indoor Positioning and Indoor Navigation (IPIN)*, 2022, pp. 1–8.
- [5] Z. Wu, X. Wu, and Y. Long, "Multi-level federated graph learning and self-attention based personalized Wi-Fi indoor fingerprint localization," *IEEE Commun. Lett.*, vol. 26, no. 8, pp. 1794–1798, 2022.
- [6] W. Yan, D. Jin, Z. Lin, and F. Yin, "Graph neural network for large-scale network localization," in *Proc. IEEE Int. Conf. Acoust. Speech and Signal Process. (ICASSP)*, 2021, pp. 5250–5254.
- [7] X. Tang and J. Yan, "A CSI Amplitude-phase Information Based Graph Construction for GNN Localization," in *Proc. Int. Conf. Comput. Inf. Telecommun. Syst. (CITS)*, 2023, pp. 01–07.
- [8] W. Kang and Y. Han, "SmartPDR: Smartphone-based pedestrian dead reckoning for indoor localization," *IEEE Sensors J.*, vol. 15, no. 5, pp. 2906–2916, 2014.
- [9] J. Yun, G. Kim, S. Ramesh, and J. Han, "RampScope: Ramp-level localization of shared mobility devices using sidewalk ramps," in *Proc. the 24th Int. Workshop Mobile Comput. Syst. Appl. (HotMobile)*, 2023, pp. 49–54.
- [10] S. Herath, H. Yan, and Y. Furukawa, "Ronin: Robust neural inertial navigation in the wild: Benchmark, evaluations, & new methods," in *Proc. IEEE Int. Conf. Robot. Automat. (ICRA)*, 2020, pp. 3146–3152.
- [11] M. Sun, Y. Wang, S. Xu, H. Qi, and X. Hu, "Indoor positioning tightly coupled Wi-Fi FTM ranging and PDR based on the extended Kalman filter for smartphones," *IEEE Access*, vol. 8, pp. 49 671–49 684, 2020.
- [12] G. Revach, N. Shlezinger, X. Ni, A. L. Escoriza, R. J. Van Sloun, and Y. C. Eldar, "KalmanNet: Neural network aided Kalman filtering for

partially known dynamics,” *IEEE Trans. Signal Process.*, vol. 70, pp. 1532–1547, 2022.

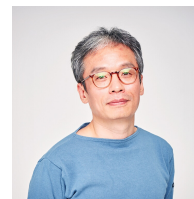
- [13] G. Choi, J. Park, N. Shlezinger, Y. C. Eldar, and N. Lee, “Split-KalmanNet: A robust model-based deep learning approach for state estimation,” *IEEE Trans. Veh. Technol.*, 2023.
- [14] S. M. Yu, K. Han, J. Park, S.-L. Kim, and S.-W. Ko, “Combinatorial data augmentation : A key enabler to bridge geometry and data-driven WiFi positioning,” *arXiv preprint arXiv:2105.07475*, 2021.
- [15] K. Han, S. M. Yu, S.-L. Kim, and S.-W. Ko, “Exploiting user mobility for WiFi RTT positioning: A geometric approach,” *IEEE Internet of Things J.*, vol. 8, no. 19, pp. 14 589–14 606, 2021.
- [16] J. Choi, “Enhanced Wi-Fi RTT ranging: A sensor-aided learning approach,” *IEEE Trans. Veh. Technol.*, vol. 71, no. 4, pp. 4428–4437, 2022.
- [17] K. Han, S. M. Yu, S.-W. Ko, and S.-L. Kim, “Waveform-guide transformation of IMU measurements for smartphone-based localization,” *IEEE Sensors J.*, vol. 23, no. 17, pp. 20 379–20 389, 2023.
- [18] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How powerful are graph neural networks?” *arXiv preprint arXiv:1810.00826*, 2018.
- [19] X. Li, L. Sun, M. Ling, and Y. Peng, “A survey of graph neural network based recommendation in social networks,” *Neurocomputing*, vol. 549, 2023.
- [20] Y. Shen, J. Zhang, S. Song, and K. B. Letaief, “Graph neural networks for wireless communications: From theory to practice,” *IEEE Trans. Wireless Commun.*, vol. 22, no. 5, pp. 3554–3569, 2022.
- [21] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” *arXiv preprint arXiv:1609.02907*, 2016.
- [22] W. Hamilton, Z. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.
- [23] T. Shim, J. Park, S.-W. Ko, S.-L. Kim, B. Lee, and J. Choi, “Traffic convexity aware cellular networks: a vehicular heavy user perspective,” *IEEE Wireless Commun.*, vol. 23, no. 1, pp. 88–94, 2016.
- [24] C. Bettstetter, G. Resta, and P. Santi, “The node distribution of the random waypoint mobility model for wireless ad hoc networks,” *IEEE Trans. Mobile Comput.*, vol. 2, no. 3, pp. 257–269, 2003.
- [25] I. Rhee, M. Shin, S. Hong, K. Lee, S. J. Kim, and S. Chong, “On the levy-walk nature of human mobility,” *IEEE/ACM Trans. Netw.*, vol. 19, no. 3, pp. 630–643, 2011.
- [26] M. D. Soltani, A. A. Purwita, Z. Zeng, C. Chen, H. Haas, and M. Safari, “An orientation-based random waypoint model for user mobility in wireless networks,” in *Proc. IEEE Int. Conf. Commun. Workshops (ICC Workshops)*, 2020, pp. 1–6.
- [27] *IEEE Standard for Information Technology–Telecommunications and Information Exchange Between Systems Local and Metropolitan Area Networks–Specific Requirements - Part 11: Wireless LAN medium access control (MAC) and physical layer (PHY) specification*, IEEE Standard 802.11-2016 (Revision of IEEE Standard 802.11-2012), 12 2016.
- [28] M. Ibrahim, H. Liu, M. Jawahar, V. Nguyen, M. Gruteser, R. Howard, B. Yu, and F. Bai, “Verification: Accuracy evaluation of wifi fine time measurements on an open platform,” in *Proc. 24th Annu. Int. Conf. Mobile Comput. Netw. (MobiCom)*, 2018, pp. 417–427.
- [29] B. K. Horn, “Doubling the accuracy of indoor positioning: Frequency diversity,” *Sensors*, vol. 20, no. 5, p. 1489, 2020.
- [30] Y. Dong, T. Arslan, and Y. Yang, “Real-time NLOS/LOS identification for smartphone-based indoor positioning systems using WiFi RTT and RSS,” *IEEE Sensors J.*, vol. 22, no. 6, pp. 5199–5209, 2021.
- [31] B. K. Horn, “Observation model for indoor positioning,” *Sensors*, vol. 20, no. 14, p. 4027, 2020.
- [32] Y. Wu, H. Zhu, Q. Dou, and S. Tang, “A pedestrian dead-reckoning system for walking and marking time mixed movement using an SHSs scheme and a foot-mounted IMU,” *IEEE Sensors J.*, vol. 19, no. 5, pp. 1661–1671, 2018.
- [33] I. Guvenc, S. Gezici, F. Watanabe, and H. Inamura, “Enhancements to linear least squares localization through reference selection and ML estimation,” in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, 2008, pp. 284–289.
- [34] Y. Ouyang, B. Guo, X. Tang, X. He, J. Xiong, and Z. Yu, “Learning cross-domain representation with multi-graph neural network,” *arXiv preprint arXiv:1905.10095*, 2019.
- [35] W. Tang, H. Sun, J. Wang, Q. Qi, H. Chen, and L. Chen, “Cross-graph embedding with trainable proximity for graph alignment,” *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12 556–12 570, 2023.
- [36] G. Li, M. Muller, A. Thabet, and B. Ghanem, “DeepGCNs: Can GCNs go as deep as CNNs?” in *Proc. IEEE/CVF Int. Conf. Comput. Vision*, 2019, pp. 9267–9276.
- [37] G. Ciano, A. Rossi, M. Bianchini, and F. Scarselli, “On inductive–transductive learning with graph neural networks,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 2, pp. 758–769, 2021.



and the Government-funded Research Institute Director Award at the Korean Institute of Communications and Information Sciences (KICS) in 2023.



tioning, and data analysis and machine learning for transportation systems. Dr. Yu received the Student Travel Grant Award at the IEEE Global Communications Conference in 2011 and the Best Paper Award at the IEEE Vehicular Technology Conference in 2013.



IEEE Transactions on Vehicular Technology, IEEE Communications Letters, Elsevier Control Engineering Practice, Elsevier ICT Express, and Journal of Communications and Network. He served as a guest editor of IEEE Wireless Communications and IEEE Network for wireless communications in networked robotics, and IEEE Journal on Selected Areas in Communications. His research interests include radio resource management, AI/ML in wireless networks, collective intelligence, and robotic networks. He is a co-recipient of IEEE VTC best paper award, IEEE Dyspan best demo award, and more recently, IEEE Heinrich Hertz Award.



a Post-Doctoral Researcher with Yonsei University, South Korea in 2015, and The University of Hong Kong (HKU) from 2016 to 2019. His research interests include intelligent wireless communications and networking for 6G, with particular emphasis on semantic communications, integrated sensing and communications, and radio-based positioning.

Kyuwon Han (S’21) received the B.S. degrees in physics from Chung-Ang University, South Korea in 2018, and the Ph.D. degree in Electrical and Electronic Engineering from Yonsei University, South Korea in 2024. He is currently working with the Advanced R&D Team at Samsung Electronics, South Korea. His research interests include indoor positioning, spectrum sharing, coverage optimization, and deep learning techniques for wireless communications. Dr. Han received a bronze prize at IEEE Seoul Section Student Paper Award in 2019

Seung Min Yu received the B.S. (Hons.) and Ph.D. degrees in Electrical and Electronic Engineering from Yonsei University, South Korea, in 2009 and 2013, respectively. He is currently a Senior Researcher with the Korea Railroad Research Institute, South Korea. He was a Senior Engineer with Networks Business, Samsung Electronics Company, Ltd., South Korea. He was also a Researcher with the Internet Research Division, NAVER Corporation (South Korea’s most popular search engine), South Korea. His research interests include indoor posi-

Seung-Lyun Kim is a Professor at the School of Electrical & Electronic Engineering, Yonsei University, Seoul, Korea. He recently led the Smart Factory Committee of 5G Forum, Korea. He was an Assistant/Associate Professor of Radio Communication Systems, Royal Institute of Technology (KTH), Stockholm, Sweden. He had visiting positions in Helsinki University of Technology (now Aalto), Finland, the KTH Center for Wireless Systems, and the Graduate School of Informatics, Kyoto University, Japan. He served as an editorial board member of